PRFs from PRGs, and Toward Public-Key Cryptography: Groups, Discrete Log, and Diffie–Hellman

CS 171: Cryptography

Sanjam Garg

October 13, 2026

Overview

1. Getting Beyond One Bit
2. PRFs from PRGs
3. Toward Public-Key Cryptography
4. Discrete Log and Diffie–Hellman
5. Diffie–Hellman Key Exchange
6. Summary

Recall: A PRG from a One-Way Permutation

- From a OWP f and its Hardness Concentration Bit:
  $\mathrm{PRG}_1(x, r) = f(x) \,\|\, r \,\|\, \langle x, r \rangle$.
- *Proved*: a distinguisher for PRG_1 becomes a predictor for $\langle x, r \rangle$.
- But it stretches by only **one bit**.

Today

Stretch to any polynomial length, then build a **PRF** from a length-doubling PRG — and with it, everything in private-key cryptography from OWPs.

From One Extra Bit to Two

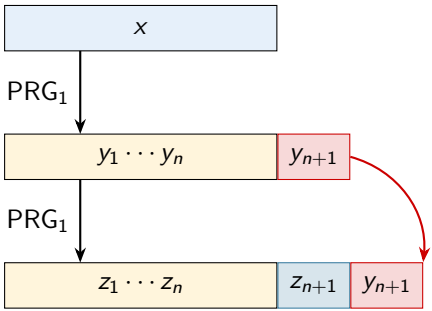
Let $\mathrm{PRG}_1 : \{0,1\}^n \to \{0,1\}^{n+1}$ be a PRG with one bit of stretch. Build $\mathrm{PRG}_2 : \{0,1\}^n \to \{0,1\}^{n+2}$:

$\mathrm{PRG}_2(x_1 \cdots x_n)$

1. $y_1 \cdots y_n \, y_{n+1} := \mathrm{PRG}_1(x_1 \cdots x_n)$.
2. $z_1 \cdots z_n \, z_{n+1} := \mathrm{PRG}_1(y_1 \cdots y_n)$.
3. Output $z_1 \cdots z_n \, z_{n+1} \, y_{n+1}$.

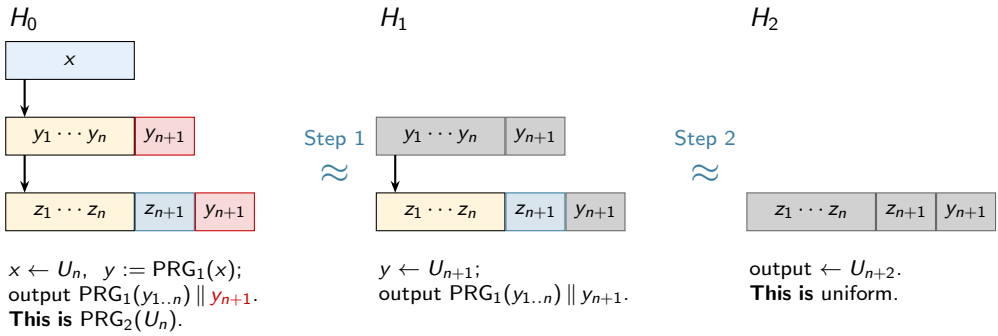
Idea

Set one bit aside; the other n bits look like a fresh seed, so stretch them **again**.



PRG_2 Is a PRG: Three Hybrids

Replace the two PRG_1 calls by **truly uniform** strings (grey), **one call at a time**:

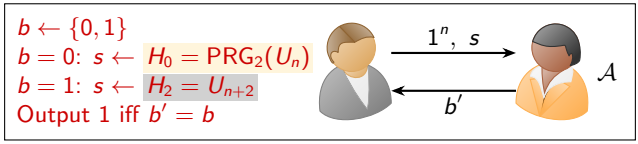


The PRG game for PRG_2 asks $\mathcal{A}$ to tell H_0 from H_2. Each step changes **one** PRG_1 output into a uniform string.

PRG_2 Is a PRG: Suppose Not

Suppose some PPT $\mathcal{A}$ has $\Pr\left[\mathrm{PRG}_{\mathcal{A},\mathrm{PRG}_2}(n) = 1\right] \geq \frac{1}{2} + \epsilon(n)$ for a non-negligible ϵ.

$\mathrm{PRG}_{\mathcal{A},\mathrm{PRG}_2}(n)$



Write $\Pr[\mathcal{A}(H) = 1]$ for the probability that $\mathcal{A}$ outputs 1 on 1^n and $s \leftarrow H$. Since b is uniform:

$$\begin{aligned}\Pr\left[\mathrm{PRG}_{\mathcal{A},\mathrm{PRG}_2}(n) = 1\right] &= \tfrac{1}{2}\Pr[\mathcal{A}(H_0) = 0] + \tfrac{1}{2}\Pr[\mathcal{A}(H_2) = 1] \\ &= \tfrac{1}{2}\left(1 - \Pr[\mathcal{A}(H_0) = 1]\right) + \tfrac{1}{2}\Pr[\mathcal{A}(H_2) = 1] \\ &= \tfrac{1}{2} + \tfrac{1}{2}\left(\Pr[\mathcal{A}(H_2) = 1] - \Pr[\mathcal{A}(H_0) = 1]\right).\end{aligned}$$

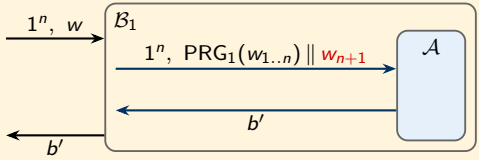
So $\mathcal{A}$ says 1 non-negligibly more often on H_2 than on H_0

$$\Pr[\mathcal{A}(H_2) = 1] - \Pr[\mathcal{A}(H_0) = 1] \ \geq\ 2\epsilon(n).$$

Each Step Is a Reduction to PRG_1

Each $\mathcal{B}_j$ plays the PRG game for PRG_1, builds a hybrid from its challenge w, gives it to $\mathcal{A}$, and outputs $\mathcal{A}$'s guess b'.

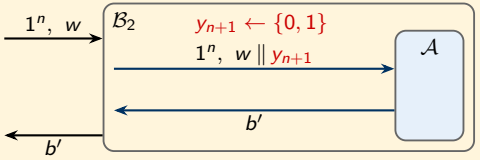
$\mathcal{B}_1$ (Step 1): its challenge w plays y



$b = 0$, $w \leftarrow \mathrm{PRG}_1(U_n)$: $\mathcal{A}$ gets H_0.
$b = 1$, $w \leftarrow U_{n+1}$: $\mathcal{A}$ gets H_1. Same calculation as for $\mathcal{A}$:

$$\Pr\left[\mathrm{PRG}_{\mathcal{B}_1,\mathrm{PRG}_1}(n) = 1\right] = \tfrac{1}{2} + \tfrac{1}{2}\left(\Pr[\mathcal{A}(H_1) = 1] - \Pr[\mathcal{A}(H_0) = 1]\right).$$

$\mathcal{B}_2$ (Step 2): its challenge w plays z



$b = 0$, $w \leftarrow \mathrm{PRG}_1(U_n)$: $\mathcal{A}$ gets H_1.
$b = 1$, $w \leftarrow U_{n+1}$: $\mathcal{A}$ gets H_2. Same calculation as for $\mathcal{A}$:

$$\Pr\left[\mathrm{PRG}_{\mathcal{B}_2,\mathrm{PRG}_1}(n) = 1\right] = \tfrac{1}{2} + \tfrac{1}{2}\left(\Pr[\mathcal{A}(H_2) = 1] - \Pr[\mathcal{A}(H_1) = 1]\right).$$

PRG_2: Putting It Together

From $\mathcal{A}$: $\Pr[\mathcal{A}(H_2) = 1] - \Pr[\mathcal{A}(H_0) = 1] \geq 2\epsilon(n)$. Add and subtract $\Pr[\mathcal{A}(H_1) = 1]$:

$$\begin{aligned} 2\epsilon(n) &\leq \Pr[\mathcal{A}(H_2) = 1] - \Pr[\mathcal{A}(H_0) = 1] \\ &= \underbrace{\big(\Pr[\mathcal{A}(H_1) = 1] - \Pr[\mathcal{A}(H_0) = 1]\big)}_{\text{Step 1}} + \underbrace{\big(\Pr[\mathcal{A}(H_2) = 1] - \Pr[\mathcal{A}(H_1) = 1]\big)}_{\text{Step 2}}. \end{aligned}$$

Two numbers that add up to at least $2\epsilon(n)$: **at least one** of them is $\geq \epsilon(n)$ (even if the other is negative). Say Step j. By the previous slide,

$$\Pr\big[\mathrm{PRG}_{\mathcal{B}_j, \mathrm{PRG}_1}(n) = 1\big] \;=\; \tfrac{1}{2} + \tfrac{1}{2} \cdot (\text{Step } j) \;\geq\; \tfrac{1}{2} + \tfrac{\epsilon(n)}{2},$$

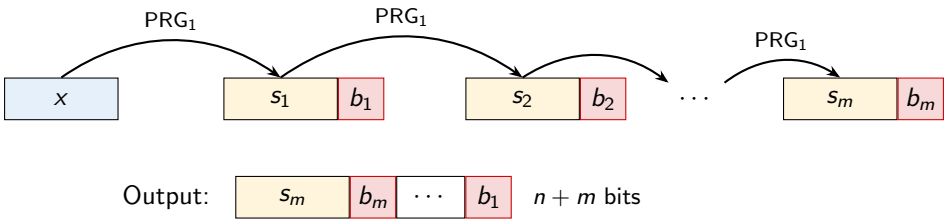
contradicting that PRG_1 is a PRG. □

Remark (running time)

Each $\mathcal{B}_j$ runs $\mathcal{A}$ once, plus one PRG_1 call or one coin flip: PPT.

Getting More Bits: PRG_m

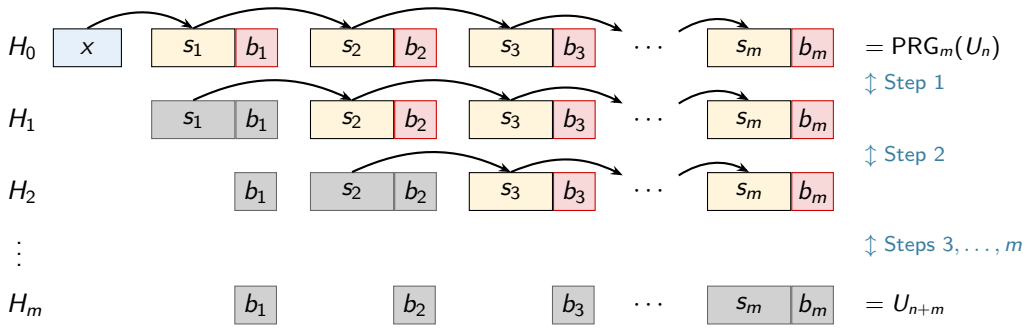
Keep going: each round sets aside one bit and re-stretches the remaining n.



With $m = n$: a **length-doubling** PRG $G : \{0,1\}^n \to \{0,1\}^{2n}$ — the starting point for PRFs, next.

PRG_m Is a PRG: $m + 1$ Hybrids

Replace the m PRG_1 calls (arrows) by **truly uniform** strings (grey), **one at a time**:



H_i: $b_1, \ldots, b_i$ and s_i uniform; rounds $i + 1, \ldots, m$ use PRG_1. Step i (H_{i-1} to H_i) changes **one** PRG_1 output, $s_i \parallel b_i$, into a uniform string.

As for PRG_2: the m step differences add up to $\geq 2\epsilon(n)$, so one of them is $\geq 2\epsilon(n)/m$ — still non-negligible, since m is polynomial in n.

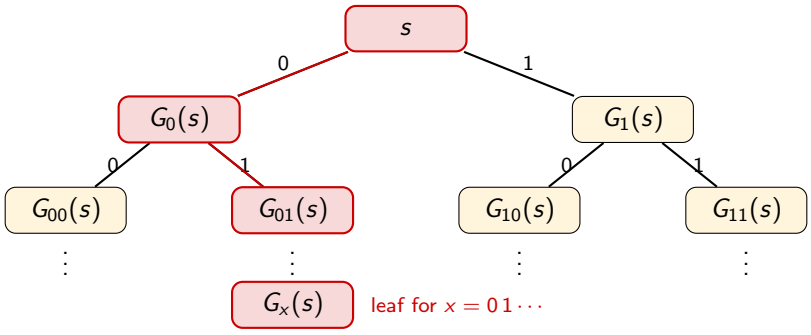
Getting PRFs: The GGM Idea

Goal

Build a PRF F — secure in the game $\mathrm{PRF}_{\mathcal{A},F}(n)$ of lecture 5 — from a length-doubling PRG. This is the GGM (Goldreich–Goldwasser–Micali) construction.

- Start from a PRG $G : \{0,1\}^n \to \{0,1\}^{2n}$ (length-doubling, e.g. PRG_n from before).
- Let $G_0(s)$ be the left half of $G(s)$, and $G_1(s)$ the right half.
- Idea: the input x chooses a root-to-leaf path in a binary tree of G applications.

The GGM Tree



Construction: $F_s(x) := G_x(s)$

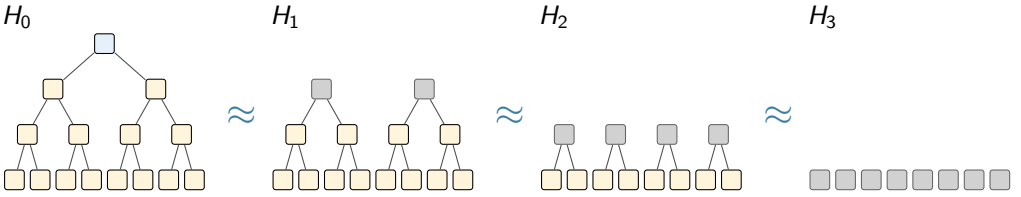
For $x = x_1 \cdots x_n$: $G_x(s) := G_{x_n}\big(\cdots G_{x_2}(G_{x_1}(s))\cdots\big)$. Each step is one PRG call: n calls per evaluation.

GGM Is a PRF: Hybrids, Level by Level

Theorem 1

If G is a length-doubling PRG, then $F_s(x) = G_x(s)$ is a PRF.

Replace the tree by truly uniform labels (grey), **one level at a time**; here $n = 3$:

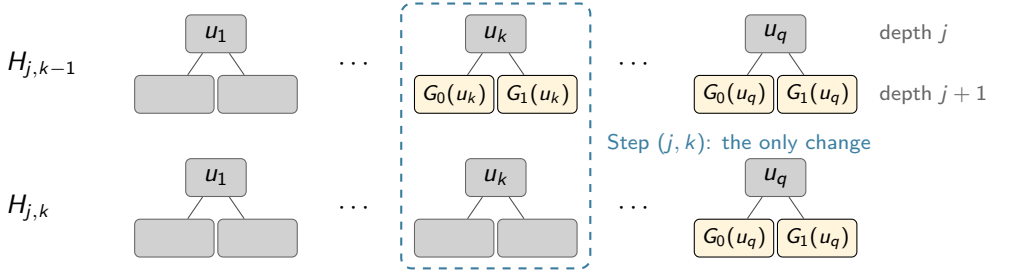


In general: hybrids $H_0, H_1, \ldots, H_n$

H_j: each **depth-j** node gets a fresh uniform label when a query first reaches it; G below. The game $\mathrm{PRF}_{\mathcal{A},F}(n)$ asks $\mathcal{A}$ to tell oracle H_0 from oracle H_n.

GGM: From H_j to H_{j+1}, One Node at a Time

$\mathcal{A}$'s q queries reach at most q depth-j nodes; call their (uniform) labels $u_1, \ldots, u_q$, in the order first reached. In $H_{j,k}$, the children of $u_1, \ldots, u_k$ are **uniform**; those of every later u_i are $G(u_i)$.

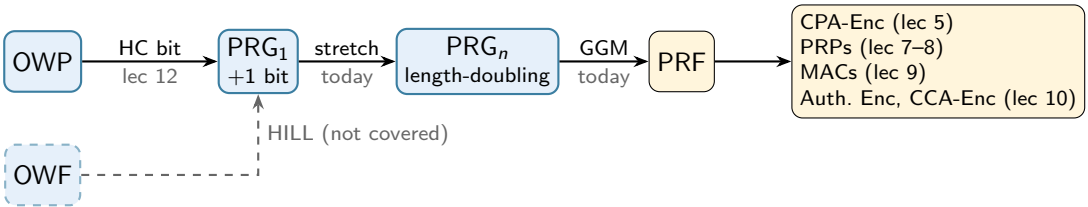


$H_{j,0} = H_j$ and $H_{j,q} = H_{j+1}$. Step (j,k) changes **one** G output, $G(u_k)$, into a uniform string.

Counting

n levels $\times$ q nodes $= nq$ steps, whose differences add up to $\geq 2\epsilon(n)$. So one step is $\geq 2\epsilon(n)/(nq)$: non-negligible, since q is polynomial in n.

Wrap-Up: Private-Key Cryptography from One-Way Functions



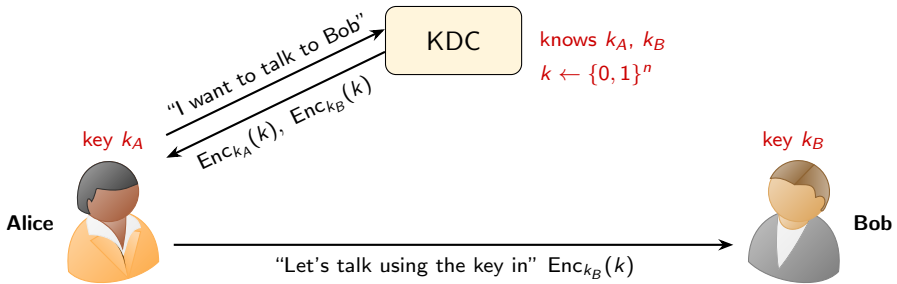
Drawbacks of Private-Key Cryptography

- **Key distribution is a problem.** Every pair of parties must *already* share a secret key, established over a secure channel before any communication.
- **Storing a large number of keys is problematic.** A network of n users who all wish to communicate needs $\binom{n}{2}$ distinct keys; each user stores $n-1$ keys.
- **Inapplicable to open systems.** Two parties who have never met (e.g. a browser and a web server) cannot communicate securely.

Goal

Secure communication *without* a pre-shared secret key.

A Partial Solution: Key-Distribution Center



- Each user shares one long-term key with the KDC: $O(n)$ keys instead of $O(n^2)$.
- But the KDC must be **online** for every session and **fully trusted** (it knows every key k).

Public-Key Cryptography

The big idea (Diffie–Hellman, 1976)

Each party has a **public key** (published) and a **secret key** (kept private). Knowing the public key lets anyone interact securely with its owner.

Plan

First some **number theory** (the group $\mathbb{Z}_p^*$, discrete log, Diffie–Hellman), then **key exchange**.

Cryptographic Group

The group $\mathbb{Z}_p^*$

For a prime p: $\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}$, with $a \cdot b := ab \bmod p$. Every element has an inverse (as p is prime), so $\mathbb{Z}_p^*$ is a **group**, of **order** (size) $p-1$.

- For $g \in \mathbb{Z}_p^*$, its powers $\langle g \rangle := \{g, g^2, g^3, \ldots\}$ form a group, **cyclic** with **generator** g.
- The **order** of g is the smallest $i > 0$ with $g^i = 1$: the size of $\langle g \rangle$.

Setup

Let p and q be primes with $p - 1 = 2q$, and let $g \in \mathbb{Z}_p^*$ be an element of order q. Then $G = \langle g \rangle$ is a cyclic group of **prime** order q.

Example 2 ($p = 23$ and $q = 11$)

$\mathbb{Z}_{23}^* = \{1, 2, \ldots, 22\}$, $\quad a \cdot b := ab \bmod 23$.

Cyclic Subgroups of $\mathbb{Z}_{23}^*$

$\langle g \rangle$ for some $g \in \mathbb{Z}_{23}^*$

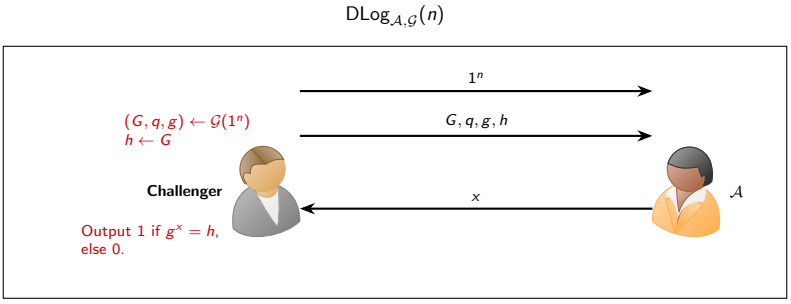
- $\langle 1 \rangle = \{1\}$ — order 1
- $\langle 2 \rangle = \{2, 4, 8, 16, 9, 18, 13, 3, 6, 12, \underbrace{2^{11}}_{=1}\}$ — order 11
- $\langle 5 \rangle = \{5, 2, 10, 4, 20, 8, 17, 16, 11, 9, \ldots, 5^{22} = 1\}$ — order 22: all of $\mathbb{Z}_{23}^*$
- $\langle 22 \rangle = \{22, 22^2 = 1\}$ — order 2 $(22 = -1)$

- Every order divides $22 = |\mathbb{Z}_{23}^*|$. Pick **any** $g \neq 1$ with $g^{11} = 1$: e.g. $G = \langle 2 \rangle$ has prime order 11. **This is the group we want.**
- For hardness, use **large** primes.

The Discrete-Log Problem: Group Generator

- Let $\mathcal{G}$ be a PPT algorithm that on input 1^n outputs the description of a **cyclic** group G, its order q (with $|q| = n$), and a generator g.
- Each element has a **unique bit representation**, and the group operation is computable in time polynomial in n.
- **Sampling a uniform group element**: sample $x \leftarrow \mathbb{Z}_q$ and compute g^x.

$\mathrm{DLog}_{\mathcal{A},\mathcal{G}}$: Challenger vs. Adversary



Remark

Computing g^x from x is easy (repeated squaring); going back from h to x is assumed hard. So $x \mapsto g^x$ is a candidate **one-way function** (lecture 11).

The Discrete-Log Problem

Game $\mathrm{DLog}_{\mathcal{A},\mathcal{G}}(n)$

1. $(G, q, g) \leftarrow \mathcal{G}(1^n)$, and $h \leftarrow G$ uniform.
2. $\mathcal{A}$ is given 1^n and (G, q, g, h), and outputs $x \in \mathbb{Z}_q$.
3. Output 1 if $g^x = h$, and 0 otherwise.

Definition 3 (Discrete-log hardness)

The discrete-log problem is **hard relative to** $\mathcal{G}$ if for all PPT $\mathcal{A}$:

$$\Pr\left[\mathrm{DLog}_{\mathcal{A},\mathcal{G}}(n) = 1\right] \leq \mathrm{negl}(n).$$

Collision-Resistant Hash Functions from DLog

Let $\mathcal{G}$ output groups of **prime** order q.

Construction (Gen, H)

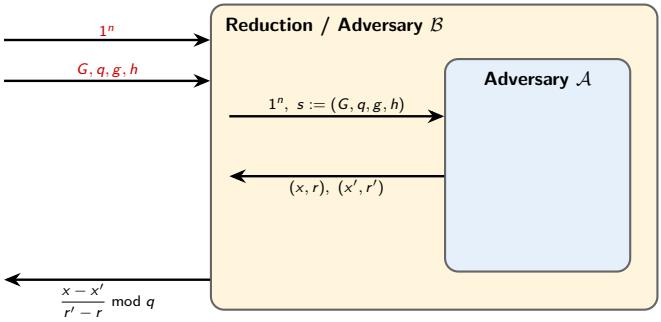
- $\mathrm{Gen}(1^n)$: $(G, q, g) \leftarrow \mathcal{G}(1^n)$; sample a uniform $h \in G$; output the key $s = (G, q, g, h)$.
- $H^s(x, r) := g^x h^r$, for $x, r \in \mathbb{Z}_q$.

- Maps two exponents $(x, r) \in \mathbb{Z}_q \times \mathbb{Z}_q$ to one group element: it compresses when group elements have about n bits (e.g. elliptic curves).
- The key s is public, as in Hash-coll (lecture 11). Note that nobody needs to know $\log_g h$.

Theorem 4

If the discrete-log problem is hard relative to $\mathcal{G}$, then (Gen, H) is collision resistant.

Proof by Reduction (If DLog Is Hard, Then CRHF)



Given / To prove

Given: $\Pr\left[\text{Hash-coll}_{\mathcal{A},(\text{Gen},H)}(n) = 1\right] \geq \epsilon(n)$ for non-negligible ϵ.
To prove: $\Pr\left[\text{DLog}_{\mathcal{B},\mathcal{G}}(n) = 1\right] \geq \epsilon(n)$.

CRHF from DLog: Putting It Together

The key s that $\mathcal{B}$ gives $\mathcal{A}$ is distributed exactly as in Gen. Suppose $\mathcal{A}$ outputs a collision: $(x, r) \neq (x', r')$ and

$$g^x h^r = g^{x'} h^{r'}.$$

- **Can $r = r'$?** Then $g^x = g^{x'}$, so $x = x' \bmod q$ — not a collision. So $r \neq r'$.
- Then $g^{x-x'} = h^{r'-r}$, and $r' - r$ is invertible modulo the **prime** q:

$$h = g^{(x-x')(r'-r)^{-1} \bmod q}.$$

Success probability of $\mathcal{B}$

$\mathcal{B}$ outputs $\log_g h$ whenever $\mathcal{A}$ wins: $\Pr\left[\mathrm{DLog}_{\mathcal{B},\mathcal{G}}(n) = 1\right] \geq \epsilon(n)$, contradicting the hardness of discrete log. □

The Diffie–Hellman Problems

Fix a cyclic group G of order q with generator g.

- **Computational (CDH)**: given g^a and g^b, compute g^{ab}.
- **Decisional (DDH)**: given g^a and g^b, distinguish g^{ab} from a uniform group element.

Computational Diffie–Hellman Problem

Game $\mathsf{CDH}_{\mathcal{A},\mathcal{G}}(n)$

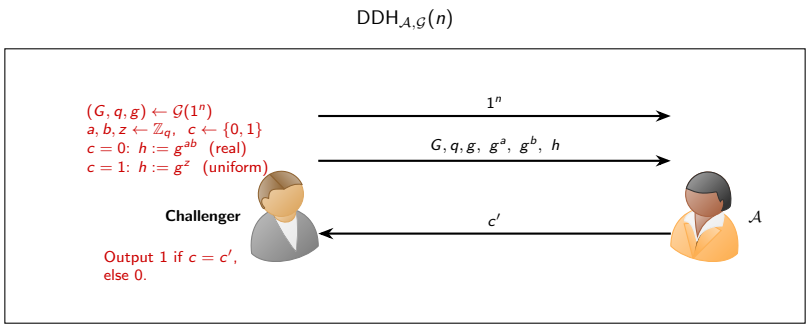
1. $(G, q, g) \leftarrow \mathcal{G}(1^n)$, and $a, b \leftarrow \mathbb{Z}_q$.
2. $\mathcal{A}$ is given 1^n and (G, q, g, g^a, g^b), and outputs $h \in G$.
3. Output 1 if $h = g^{ab}$, and 0 otherwise.

Definition 5 (CDH hardness)

CDH is **hard relative to** $\mathcal{G}$ if for all PPT $\mathcal{A}$:

$$\Pr\left[\mathsf{CDH}_{\mathcal{A},\mathcal{G}}(n) = 1\right] \leq \mathsf{negl}(n).$$

$\mathrm{DDH}_{\mathcal{A},\mathcal{G}}$: Challenger vs. Adversary



Decisional Diffie–Hellman Problem

Game $\mathrm{DDH}_{\mathcal{A},\mathcal{G}}(n)$

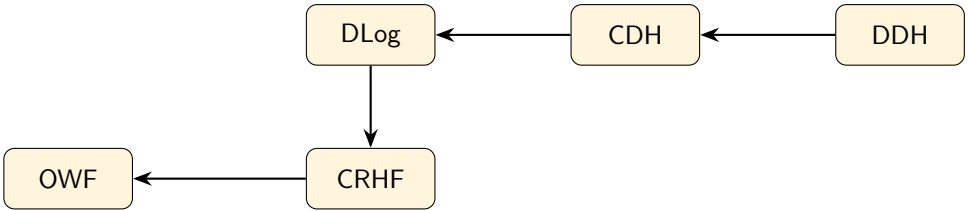
1. $(G, q, g) \leftarrow \mathcal{G}(1^n)$, and $a, b, z \leftarrow \mathbb{Z}_q$. Sample a uniform bit c.
2. If $c = 0$ set $h := g^{ab}$ (real); if $c = 1$ set $h := g^z$ (uniform).
3. $\mathcal{A}$ is given 1^n and (G, q, g, g^a, g^b, h), and outputs c'.
4. Output 1 if $c' = c$, and 0 otherwise.

Definition 6 (DDH hardness)

DDH is **hard relative to** $\mathcal{G}$ if for all PPT $\mathcal{A}$:

$$\Pr\left[\mathrm{DDH}_{\mathcal{A},\mathcal{G}}(n) = 1\right] \leq \frac{1}{2} + \mathrm{negl}(n).$$

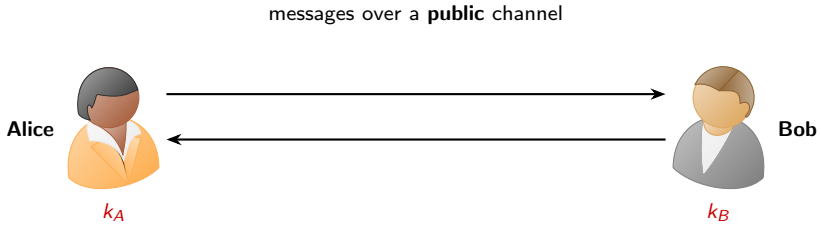
Diffie–Hellman Problems: Relations



$X \to Y$: "if X is hard, then Y is hard". DDH is the **strongest** assumption.

- **DDH** $\to$ **CDH**: a CDH solver breaks DDH — compute g^{ab} and compare it with h.
- **CDH** $\to$ **DLog**: a DLog solver breaks CDH — find a from g^a, output $(g^b)^a = g^{ab}$.
- **DLog** $\to$ **CRHF**: the theorem we just proved. **CRHF** $\to$ **OWF**: a collision-resistant (compressing) hash function is one-way.

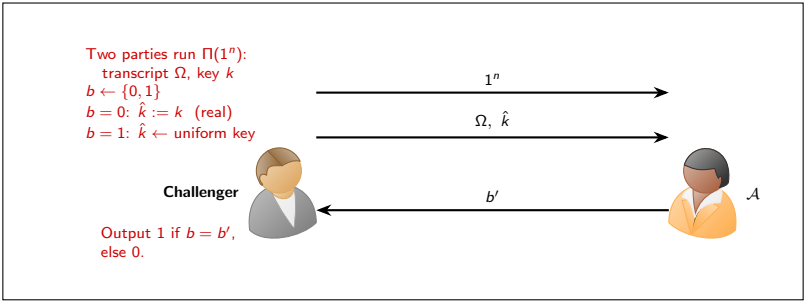
Key Exchange



- **Correctness**: $k_A = k_B =: k$.
- **Security (informally)**: an eavesdropper who sees all the messages should not be able to tell k apart from a random key.

$\mathsf{KE}^{\mathsf{eav}}_{\mathcal{A},\Pi}$: Challenger vs. Adversary

$\mathsf{KE}^{\mathsf{eav}}_{\mathcal{A},\Pi}(n)$



Key Exchange: Security

Game $\mathsf{KE}^{\mathsf{eav}}_{\mathcal{A},\Pi}(n)$

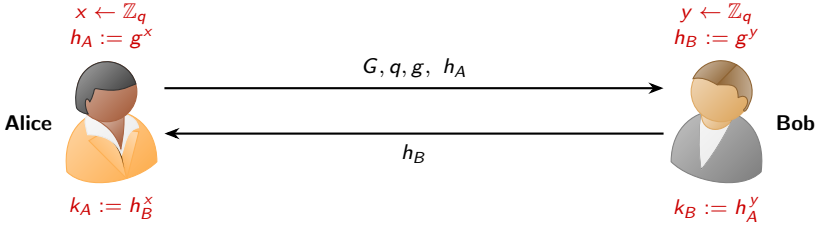
1. Two parties holding 1^n execute Π. This gives a transcript Ω of the messages and a key k output by each party.
2. Sample a uniform bit b. If $b = 0$ set $\hat{k} := k$; if $b = 1$ sample $\hat{k}$ uniformly.
3. $\mathcal{A}$ is given 1^n and $(\Omega, \hat{k})$, and outputs b'.
4. Output 1 if $b' = b$, and 0 otherwise.

Definition 7 (Secure key exchange)

A key-exchange protocol Π is **secure** (against eavesdroppers) if for all PPT $\mathcal{A}$:

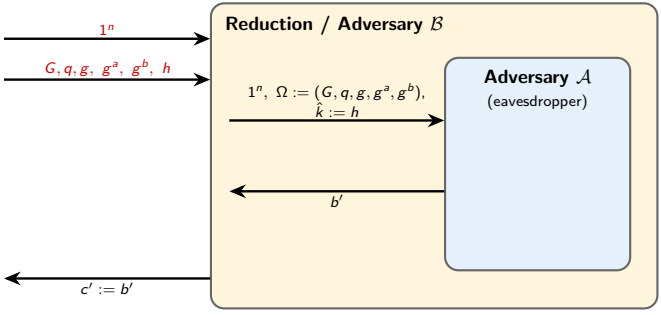
$$\Pr\left[\mathsf{KE}^{\mathsf{eav}}_{\mathcal{A},\Pi}(n) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(n).$$

The Diffie–Hellman Key-Exchange Protocol



- G, q, g can be fixed once and for all.
- **Correctness**: $k_A = (g^y)^x = g^{xy} = (g^x)^y = k_B$.
- **Security**: follows from the **DDH** assumption (next slide).
- **Subtle point**: the key is indistinguishable from a uniform *group element*, not from a uniform *bit string*.

Diffie–Hellman Is Secure: Proof by Reduction



Why it works

(G, q, g, g^a, g^b) is exactly a DH transcript with $x = a$, $y = b$, whose key is g^{ab}. $c = 0$: $\hat{k}$ is the real key ($b = 0$). $c = 1$: $\hat{k}$ is a uniform group element ($b = 1$). So $\Pr\left[\mathrm{KE}^{\mathsf{eav}}_{\mathcal{A},\Pi}(n) = 1\right] = \Pr\left[\mathrm{DDH}_{\mathcal{B},\mathcal{G}}(n) = 1\right] \leq \frac{1}{2} + \mathrm{negl}(n)$. □

Summary

- **Stretch**: $\mathrm{PRG}_1 \Rightarrow \mathrm{PRG}_m$ for any polynomial m, proved by hybrids.
- **GGM**: length-doubling PRG $\Rightarrow$ PRF. So OWPs give all of private-key crypto.
- **Public-key crypto**: no pre-shared keys; built on groups such as $\mathbb{Z}_p^*$.
- **Hardness**: DDH $\Rightarrow$ CDH $\Rightarrow$ DLog $\Rightarrow$ collision-resistant hashing.
- **Diffie–Hellman key exchange**: shared key g^{xy}, secure under DDH.